

PONTIFÍCIA UNIVERSIDADE CATÓLICA DE GOIÁS

Escola Politécnica e de Artes | Análise e Desenvolvimento de Sistemas

ATIVIDADE INTEGRADA - PI

Projeto de Aplicação Web Full-Stack

DISCIPLINA 1
Desenvolvimento Web

DISCIPLINA 2
Modelagem de Interfaces — UI/UX

DISCIPLINA 3
Design de Software

 **Prazo de desenvolvimento: 1 semana**
Grupos de até 4 integrantes | Tema livre

1. Visão Geral da Atividade

Esta atividade integrada articula três disciplinas do Curso de Análise e Desenvolvimento de Sistemas da PUC Goiás — Desenvolvimento Web, Modelagem de Interfaces UI/UX e Design de Software — em um único projeto prático com prazo de uma semana.

O objetivo é simular um ciclo curto de desenvolvimento próximo à realidade profissional: partindo de um problema escolhido pelo grupo, os alunos devem projetar, implementar e apresentar uma aplicação Web full-stack funcional, coerente do ponto de vista técnico, visual e arquitetural.

1.1. Contexto e Justificativa

Ao trabalhar com prazo reduzido, o grupo é forçado a tomar decisões rápidas de escopo, priorizar funcionalidades e evitar desperdício de esforço. Essa é, na prática, a dinâmica de sprints curtos em times de produto. A integração entre as três disciplinas reforça que decisões de back-end, de interface e de arquitetura não ocorrem em silos: um padrão de projeto bem aplicado facilita a integração com a API; um Design System adotado corretamente acelera a entrega do front-end; uma autenticação bem estruturada protege toda a aplicação.

1.2. Formação dos Grupos e Tema

- Grupos de até 4 integrantes, formados pelos próprios alunos.
- O tema da aplicação é livre.
- A aplicação deve ter contexto claro: qual problema resolve e para quem.
- Temas iguais entre grupos diferentes não são permitidos.
- A aplicação deve contemplar ao menos dois perfis de usuário (ex.: administrador e usuário comum).

1.4. Requisitos Funcionais Mínimos

Independentemente do tema, toda aplicação deve ter:

1. Autenticação com pelo menos dois perfis de acesso distintos.
2. CRUD completo de ao menos uma entidade principal do domínio.
3. Listagem de registros com algum tipo de filtro ou busca.
4. Feedback visual para ações do usuário: sucesso, erro e carregamento.
5. Interface responsiva: funcionar em mobile e desktop.
6. Validação de formulários no front-end e no back-end.

DISCIPLINA 1
Desenvolvimento Web

2. Desenvolvimento Web

2.1. Visão da Disciplina

A disciplina foca na construção full-stack da aplicação. O grupo implementa o back-end em Node.js, expondo uma API RESTful com autenticação JWT e persistência em banco de dados, e o front-end em HTML, CSS e JavaScript puro, integrado à API via requisições HTTP.

2.2. Back-End com Node.js

2.2.1. Servidor e API REST

- Node.js com Express.js como framework HTTP.
- Código organizado em camadas: rotas → controladores → serviços → modelos.
- Verbos HTTP usados corretamente: GET, POST, PUT/PATCH, DELETE.
- Respostas em JSON com status HTTP adequados (200, 201, 400, 401, 403, 404, 500).
- Middleware de tratamento centralizado de erros.
- Credenciais e configurações sensíveis em variáveis de ambiente via arquivo .env (nunca versionado).

2.2.2. Autenticação e Autorização com JWT

- Registro e login de usuários com senha armazenada com hash (bcrypt).
- Geração de JWT no login contendo id e perfil (role) do usuário no payload.
- Middleware de autenticação verificando o token em todas as rotas protegidas.
- Middleware de autorização bloqueando acesso a recursos restritos por perfil.
- Token com tempo de expiração configurável via .env.

2.2.3. Persistência de Dados

- Banco relacional (PostgreSQL, MySQL ou SQLite) ou NoSQL (MongoDB).
- ORM/ODM (Prisma, Sequelize, Mongoose ou equivalente) ou queries parametrizadas.
- Relacionamentos modelados adequadamente ao domínio escolhido.
- Script de seed para popular o banco com dados iniciais para demonstração.

2.3. Front-End

2.3.1. HTML Semântico

- Tags semânticas HTML5: header, main, nav, section, article, footer, aside.
- Formulários com labels associados e atributos de validação nativos.

- Hierarquia de headings respeitada (h1 → h2 → h3).

2.3.2. CSS e Responsividade

- Layout com Flexbox e/ou CSS Grid.
- Responsivo para mobile ($\leq 768\text{px}$) e desktop ($> 1024\text{px}$).
- Variáveis CSS (custom properties) para cores, espaçamentos e tipografia — preferencialmente sincronizadas com os tokens do Design System adotado.

2.3.3. JavaScript e Integração com a API

- Comunicação com o back-end via Fetch API ou Axios.
- JWT armazenado no localStorage ou sessionStorage; enviado como Bearer Token no header Authorization.
- Redirecionamento para login quando o token estiver ausente ou expirado.
- Renderização dinâmica de listas, feedbacks e estados da interface via manipulação de DOM.

2.4. Boas Práticas e Infraestrutura

- Repositório no GitHub com histórico de commits semânticos (feat, fix, refactor, docs...).
- README com instruções claras de instalação, configuração do .env e execução local.
- .gitignore cobrindo node_modules, .env e arquivos de build.
- Aplicação inicializável com um único comando após npm install.

2.5. Entregas — Desenvolvimento Web

Artefato	Descrição / Critério de aceite
Repositório no GitHub	Código completo, histórico de commits visível, README detalhado.
Coleção Postman ou Insomnia	Todas as rotas documentadas com exemplos de request e response.
Diagrama ER / de coleções	Modelo de dados com entidades, atributos e relacionamentos.
Aplicação rodando localmente	Deve funcionar sem erros com npm install && npm start.
Apresentação ao vivo	Demonstração no browser + arguição sobre decisões técnicas.

2.6. Critérios de Avaliação — Desenvolvimento Web

Critério	Peso
API RESTful funcional e estruturada em camadas	25%
Autenticação e autorização com JWT	20%
Persistência e modelagem de dados	20%
Front-end integrado e funcional	20%
Boas práticas, Git e documentação	15%

DISCIPLINA 2

Modelagem de Interfaces — UI/UX

3. Modelagem de Interfaces — UI/UX

3.1. Visão da Disciplina

A disciplina foca na qualidade da experiência de uso. Com o prazo de uma semana, o grupo deve tomar decisões de interface rápidas e fundamentadas, adotando um Design System consolidado do mercado como base visual e de componentes, e aplicando boas práticas de UX no fluxo, na hierarquia e no feedback da interface.

O grupo não criará um Design System do zero. Em vez disso, escolherá e adotará consistentemente um dos sistemas listados na seção 3.2, justificando a escolha e demonstrando que os princípios do sistema foram respeitados na implementação.

3.2. Design Systems Aceitos

O grupo deve escolher um dos Design Systems abaixo (ou outro aprovado pelo professor) e utilizá-lo como fundamento visual e de componentes do projeto:

Design System	Tecnologia / Como usar
Material Design 3 (Google)	Biblioteca MUI (React) ou componentes Web via Material Web Components.
Fluent UI (Microsoft)	Pacote @fluentui/react ou @fluentui/web-components.
Ant Design	Biblioteca antd (React) ou componentes isolados via CDN.
Bootstrap 5	CDN ou npm; usar componentes e utilitários do sistema.
Tailwind CSS + shadcn/ui	Tailwind como base de tokens + componentes do shadcn/ui.
Chakra UI	Biblioteca @chakra-ui/react com sistema de temas.
Carbon Design System (IBM)	Pacote @carbon/react ou Web Components.
Outro (a combinar)	Qualquer DS documentado publicamente, com aprovação prévia do professor.

3.3. Boas Práticas de UI/UX a Aplicar

Durante o desenvolvimento, o grupo deve demonstrar consciência e aplicação das seguintes práticas:

3.3.1. Hierarquia Visual e Layout

- Hierarquia clara: o elemento mais importante de cada tela deve ser o de maior destaque visual (tamanho, cor, peso).
- Uso consistente do sistema de espaçamento do DS escolhido (sem espaçamentos arbitrários).
- Grid ou sistema de colunas respeitado em todas as telas.
- Densidade adequada: nem telas vazias demais, nem poluição visual.

3.3.2. Consistência e Padrões

- Mesmos componentes para as mesmas funções em toda a aplicação (ex.: sempre o mesmo componente de botão primário para ações principais).
- Linguagem visual uniforme: paleta de cores, tipografia e ícones do DS sem mistura com outros sistemas.
- Navegação previsível: o usuário sabe onde está e como voltar em qualquer ponto da aplicação.
- Labels e textos de interface em português consistente (sem mistura de idiomas na UI).

3.3.3. Feedback e Comunicação com o Usuário

- Estado de carregamento (loading): indicador visual sempre que houver uma operação assíncrona em andamento.
- Mensagens de sucesso: confirmação clara quando uma ação é concluída (ex.: toast, snackbar, banner).
- Mensagens de erro: linguagem amigável, explicando o que ocorreu e o que o usuário pode fazer.
- Estados vazios: tela de lista sem registros deve ter mensagem explicativa e, quando possível, ação sugerida.
- Confirmação de ações destrutivas: exclusão de dados deve exigir confirmação explícita (modal ou dialog).

3.3.4. Formulários e Validação

- Labels sempre visíveis (nunca apenas placeholder como substituto de label).
- Mensagens de erro inline, próximas ao campo com problema, e não apenas no topo do formulário.
- Botão de submit desabilitado ou com estado de loading durante o processamento.
- Campos obrigatórios claramente sinalizados.

3.3.5. Acessibilidade Mínima

- Contraste de texto vs. fundo: mínimo 4.5:1 para textos normais (WCAG AA) — verificar com a paleta do DS.
- Todos os elementos interativos acessíveis por teclado (Tab, Enter, Esc).
- Imagens decorativas com alt vazio; imagens informativas com alt descritivo.
- Não usar apenas cor como único indicador de estado (ex.: campos com erro devem ter ícone ou texto, além da cor vermelha).

3.3.6. Responsividade

- Interface funcional e legível em telas a partir de 375px de largura (iPhone SE).
- Navegação adaptada para mobile: menu hamburguer ou bottom navigation quando necessário.
- Tabelas e listas horizontais adaptadas para o modo mobile (scroll horizontal sinalizado ou layout alternativo).
- Campos de formulário com tamanho de toque mínimo de 44×44px em mobile.

3.4. Protótipo de Telas (Entrega Obrigatória)

Antes de iniciar a implementação, o grupo deve criar um protótipo das telas principais no Figma (ou ferramenta equivalente aprovada). O protótipo deve cobrir:

7. Tela de login e/ou cadastro.
8. Tela principal (dashboard ou listagem) para cada perfil de usuário.
9. Tela de cadastro/edição da entidade principal.
10. Ao menos um estado de erro e um estado vazio.

O protótipo não precisa ser de alta fidelidade completa — wireframes com indicação de componentes do DS já são suficientes. O importante é que exista uma referência visual antes da codificação.

3.5. Entregas — UI/UX

Artefato	Descrição / Critério de aceite
Link do protótipo (Figma)	Telas principais navegáveis, indicando os componentes do DS utilizados.
Justificativa do Design System	Parágrafo no relatório explicando a escolha do DS e como ele foi utilizado.
Checklist de boas práticas	Documento ou seção no relatório com cada item das seções 3.3.1 a 3.3.6 marcado como atendido ou não atendido, com justificativa.
Frontend implementado	Interface real (não o protótipo) usando componentes do DS escolhido, responsiva e com feedbacks visuais funcionando.
Apresentação ao vivo	Navegação pela aplicação real demonstrando os itens de UX + arguição sobre decisões de interface.

3.6. Critérios de Avaliação — UI/UX

Critério	Peso
Uso consistente e correto do Design System escolhido	30%
Feedback visual (loading, sucesso, erro, empty states)	25%
Hierarquia visual, layout e consistência	20%
Responsividade e acessibilidade mínima	15%
Qualidade e coerência do protótipo inicial	10%

DISCIPLINA 3

Design de Software

4. Design de Software

4.1. Visão da Disciplina

A disciplina foca na qualidade estrutural do código: como ele é organizado, quais padrões tornam o sistema extensível e manutenível, e como as decisões arquiteturais são comunicadas. Mesmo com prazo de uma semana, espera-se que o grupo aplique conscientemente ao menos dois Design Patterns e demonstre compreensão dos princípios de boa estruturação de software.

4.2. Arquitetura da Aplicação

4.2.1. Organização em Camadas

O back-end deve ser organizado em camadas com responsabilidades bem definidas:

Rotas (routes/) → Define os endpoints e associa cada um ao controlador correspondente.
Controladores (controllers/) → Recebe a requisição, delega ao serviço e retorna a resposta HTTP.
Serviços (services/) → Contém a lógica de negócio; não conhece req/res do Express.
Modelos / Repositórios (models/ ou repositories/) → Abstração do acesso ao banco de dados.
Middlewares (middlewares/) → Autenticação, autorização, validação e tratamento de erros.

4.3. Aplicação de Design Patterns

O grupo deve aplicar ao menos 2 (dois) Design Patterns distintos, documentando cada um. Padrões podem ser do catálogo Gang of Four (GoF) ou padrões arquiteturais reconhecidos.

Para cada padrão aplicado, a documentação deve conter:

1. Nome do padrão e categoria (Criacional / Estrutural / Comportamental).
2. Problema concreto que motivou o uso no projeto.
3. Como foi implementado — com trecho de código real.
4. Benefício obtido: o que ficou mais fácil de manter, estender ou testar.

Exemplos de padrões e onde costumam aparecer neste tipo de projeto:

Padrão	Contexto de uso comum neste projeto
Repository (Estrutural)	Abstração do acesso ao banco; isola a camada de serviço da tecnologia de persistência.
Singleton (Criacional)	Conexão com o banco de dados: uma única instância compartilhada em toda a aplicação.
Factory Method (Criacional)	Criação de objetos de resposta padronizados da API (ex.: <code>ApiResponse.success()</code> , <code>ApiResponse.error()</code>).
Strategy (Comportamental)	Diferentes estratégias de validação ou de envio de notificação (e-mail, SMS, push).
Decorator / Middleware (Estrutural)	Middlewares do Express como decorators: autenticação, logging, validação encadeados.
Observer / EventEmitter (Comportamental)	Notificações internas após ações (ex.: evento 'pedido.criado' dispara lógica de notificação).
Facade (Estrutural)	Simplificar a interface de um subsistema complexo expondo apenas o necessário ao controlador.
Template Method (Comportamental)	Fluxo base de importação ou exportação de dados com etapas fixas e etapas customizáveis.

4.4. Princípios SOLID — Aplicação Consciente

O grupo deve demonstrar, na documentação e no código, a aplicação de ao menos 2 princípios SOLID, com exemplos reais do projeto:

- **S — Single Responsibility:** cada módulo ou classe tem uma única razão para mudar. Ex.: serviços não manipulam objetos HTTP.
- **O — Open/Closed:** aberto para extensão, fechado para modificação. Ex.: adicionar um novo tipo de usuário sem alterar a lógica existente.
- **L — Liskov Substitution:** subtipos substituíveis pelo tipo base sem quebrar o comportamento.
- **I — Interface Segregation:** interfaces coesas e específicas. Ex.: separar contratos de leitura e escrita.
- **D — Dependency Inversion:** depender de abstrações, não de implementações. Ex.: serviço depende de interface do repositório, não da classe concreta.

4.5. Qualidade de Código

- Nomenclatura consistente e descritiva em todo o projeto (sem variáveis como `x`, `temp`, `data2`).
- Funções curtas e coesas: cada função faz uma única coisa bem definida.
- Sem código morto (funções, variáveis ou imports nunca utilizados).
- Sem números ou strings mágicos: uso de constantes nomeadas (`const MAX_ITENS_POR_PAGINA = 20`).
- Tratamento explícito de erros em todas as operações assíncronas (`try/catch` ou `.catch()`).

4.6. Entregas — Design de Software

Artefato	Descrição / Critério de aceite
Diagrama estrutural	Diagrama de classes ou estrutural representando as entidades e camadas do sistema.
Documentação dos patterns	Seção no relatório técnico descrevendo cada padrão aplicado (nome, problema, código, benefício).
Exemplos SOLID	Seção com ao menos 2 princípios SOLID identificados e exemplificados com código real.
Código anotado	Comentários JSDoc ou de bloco nos pontos onde os padrões foram aplicados.
Apresentação ao vivo	Explicação do diagrama e demonstração dos padrões no código-fonte + arguição.

4.7. Critérios de Avaliação — Design de Software

Critério	Peso
Aplicação e documentação dos Design Patterns (mín. 2)	35%
Organização em camadas e coerência arquitetural	25%
Aplicação dos princípios SOLID (mín. 2)	20%
Qualidade geral do código	20%

5. Apresentação Final

A apresentação ocorre no Dia 7 e é única para as três disciplinas. Cada grupo terá entre 20 e 30 minutos, organizados da seguinte forma:

Bloco	Conteúdo	Tempo
Demo ao vivo	Navegação pela aplicação funcionando no browser, cobrindo os fluxos principais e os dois perfis de usuário.	8 min
UI/UX	Apresentar o Design System adotado, mostrar onde e como foi aplicado, e destacar as decisões de boas práticas de interface.	6 min
Arquitetura	Mostrar o diagrama de camadas/classes e explicar os Design Patterns aplicados diretamente no código.	6 min
Arguição	Perguntas dos professores para cada integrante individualmente sobre qualquer aspecto do projeto.	5–10 min

Recomendações para a apresentação:

- Tenha a aplicação rodando localmente antes de começar — não dependa de deploy em produção.
- Popule o banco com dados realistas para a demo (use o script de seed).
- Prepare um vídeo de backup caso haja instabilidade de rede ou ambiente.
- Na arguição, cada membro é avaliado individualmente: todos devem conhecer o projeto inteiro.

6. Referências e Recursos

Desenvolvimento Web

- Documentação do Express.js: <https://expressjs.com>
- Documentação do Prisma ORM: <https://www.prisma.io/docs>
- jsonwebtoken (npm): <https://www.npmjs.com/package/jsonwebtoken>
- MDN Web Docs — HTML, CSS e JS: <https://developer.mozilla.org>

Design Systems

- Material Design 3: <https://m3.material.io>
- Ant Design: <https://ant.design>
- Bootstrap 5: <https://getbootstrap.com>
- shadcn/ui: <https://ui.shadcn.com>
- Chakra UI: <https://chakra-ui.com>
- WCAG 2.1 — Acessibilidade: <https://www.w3.org/TR/WCAG21>

Design de Software

- Refactoring Guru — Catálogo de Patterns: <https://refactoring.guru/design-patterns>
- GAMMA, E. et al. Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley, 1994.
- MARTIN, R. C. Clean Code. Prentice Hall, 2008.
- PlantUML — Diagramas em texto: <https://plantuml.com>

Bom trabalho! Uma semana é curta — foquem no essencial, entregas consistentes valem mais do que features incompletas.